AI with Rav

Learn AI the way you actually think.

DAY 7 of 30

MACHINE LEARNING

Decision Trees — the "20 Questions" algorithm

WHAT YOU'LL LEARN TODAY

- How a decision tree makes decisions
- How it picks the "best question" to ask
- Why trees are so easy to explain
- When trees overfit — and the simple fix

Decision Trees — the "20 Questions" algorithm

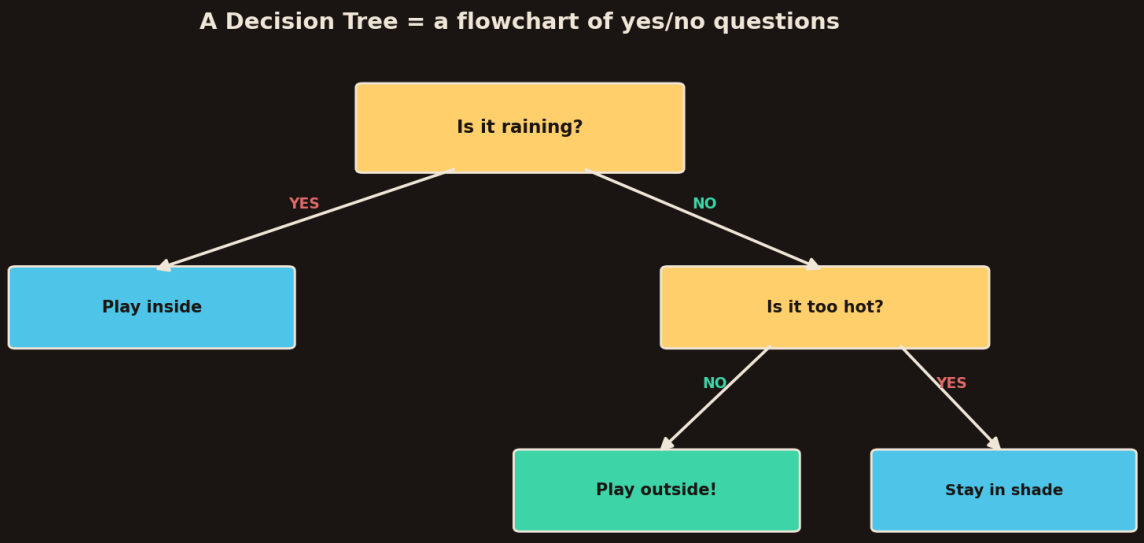
> In One Line: A Decision Tree asks a series of yes/no questions — like the game "20 Questions" — and follows the answers down to a final decision.

Start here: the 20 Questions game

You've played it: "Is it an animal? Is it bigger than a cat? Does it bark?" Each answer narrows things down until you guess right.

A **Decision Tree** is a machine playing 20 Questions. It asks smart yes/no questions about the data, and each answer sends it down a branch — until it reaches an answer. It's the **most human-like** algorithm, because you can literally read its thinking.

What a decision tree looks like



Each box asks a question; each branch is an answer; the leaves are the final decisions.

A tree of questions. Each box asks something; each branch is an answer; the leaves are the final decisions.

- Each **box** = a yes/no question about a feature ("Is it raining?").
- Each **branch** = an answer (yes / no).
- Each **leaf** (end) = a final decision ("Play inside").

> This works for both jobs: predicting a **category** (spam / not-spam — a classification tree) OR a **number** (house price — a regression tree). Same idea, different leaves.

How does it pick the "best" question?

A good question splits the data cleanly — spam mostly on one side, not-spam on the other. A bad question mixes them up.

The tree doesn't guess questions randomly. At each step it tries every possible question and picks the one that **separates the groups best**:

- A **good** split → one side is mostly "spam", the other mostly "not spam" (clean).
- A **bad** split → both sides are a messy mix (useless).

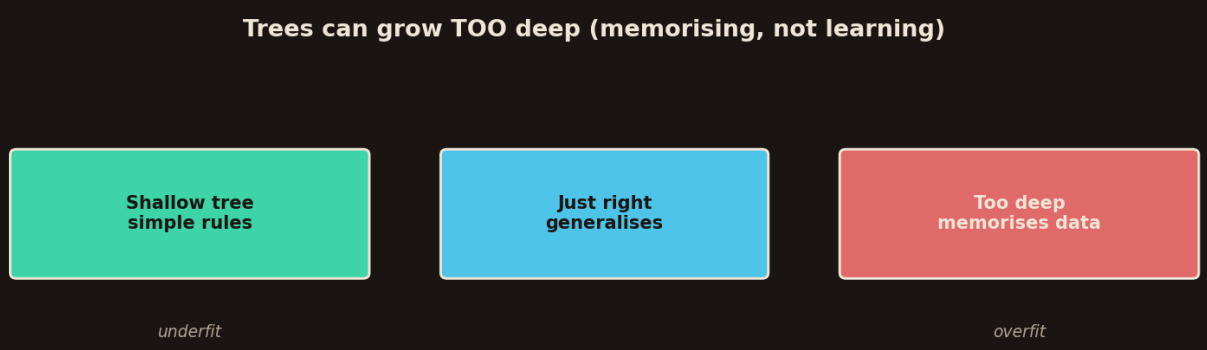
> It measures "messiness" with a score (called *Gini* or *entropy* — just fancy words for "how mixed up is this group?"). It picks the question that reduces the mess the most. That's the entire training process: keep asking the question that tidies the data best.

Why everyone loves decision trees

- **You can SEE the logic** → unlike most ML, you can print the tree and read exactly why it decided something. Huge for trust (banks, doctors, courts).
- **No fancy prep needed** → they handle numbers and categories, don't need scaling.
- **Fast and simple** → easy to train, easy to explain to your boss.

> This is the big win: a Decision Tree can explain itself. "Loan rejected because income < 30k AND no credit history." Try getting that from a neural network. This is why trees are everywhere in real business ML.

The catch: overfitting



A tree with too many questions memorises the training data and fails on new data. Fix: limit its depth.

Too shallow = underfit (too simple). Too deep = overfit (memorises). The middle is just right.

If you let a tree ask **too many** questions, it stops learning patterns and starts **memorising** the exact training data — like a student who memorises answers instead of understanding. It looks perfect on training data but fails on new data.

- Too **shallow** → too simple, misses real patterns (underfitting).
- Too **deep** → memorises noise, fails on new data (overfitting).

> The fix: limit the tree's depth (e.g. `max_depth=5`) so it can't grow endlessly. We'll go deeper on overfitting in Video 15–16 — for now, just know: a tree left unchecked will cheat by memorising.

Train one — a few lines

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=5)   # limit depth = avoid overfit
model.fit(X_train, y_train)

print(model.predict([[income, age, has_credit]]))
# you can even export and READ the whole tree:
from sklearn.tree import export_text
print(export_text(model))
```

> `export_text` prints the actual if/else rules the tree learned. No other algorithm lets you read its brain this easily.

How the tree scores a question — the fruit basket

We said the tree picks the question that "separates groups best." But how does it *measure* that? With a simple score called **Gini**. Let's understand it with a basket of fruits.

Gini is just a "mixed-up score" for a basket. All same fruit = 0 (clean). Half-and-half = 0.5 (fully mixed). A little mixed = a small number.

Imagine a basket of fruits, and you want baskets that are **all one fruit**:

- Basket of **only mangoes** → not mixed at all → **Gini = 0** (perfect, clean).
- **Half mangoes, half apples** → totally mixed → **Gini = 0.5** (the worst).
- **Mostly mangoes, one apple** → only a little mixed → small Gini (like 0.28).

> So Gini is just a "how mixed is this basket?" score. **0 means clean, 0.5 means fully mixed.** The tree's whole job: ask the question that makes the baskets as *clean* (low Gini) as possible.

The Gini formula — but really simple

Here's the formula. Don't run away — it's easier than it looks, and it's the same fruit basket:

> **Gini = 1 – (chance of picking a mango)² – (chance of picking an apple)²**

Let's just plug in our baskets:

- **All mangoes:** chance of mango = 1, apple = 0. So `1 – 1² – 0² = 0`. Clean! ✔
- **Half & half:** mango = ½, apple = ½. So `1 – (½)² – (½)² = 1 – 0.25 – 0.25 = 0.5`. Fully mixed.
- **The squaring** (the little ²) is the trick: it rewards baskets that lean heavily one way. One big fruit type → a big square → Gini drops toward 0.

That's the whole formula. "1 minus the squared chances." A clean basket scores 0; a mixed one scores higher. The tree keeps choosing questions that lower this number — it's chasing clean baskets.

So how does it choose? (still the basket)

At every step the tree tries every question and asks: *"After this split, are my two baskets cleaner than before?"* It picks the question that drops the Gini the most.

- Before the split → one messy basket (high Gini).
- After a **good** question → two cleaner baskets (lower Gini) → the tree keeps it.
- The drop in messiness has a name: **information gain** — literally "how much cleaner did we get?"

One thing to watch (important): if you let the tree keep asking questions forever, it makes a tiny basket for *every single fruit* — one fruit per basket. That's not learning, that's memorising. On a new fruit it gets confused. The fix is simple: tell it to stop early (`max_depth=5`) so it keeps sensible rules instead of memorising. (More on this in Video 15–16.)

Recap — the 20-second version

- A Decision Tree plays **"20 Questions"** — yes/no questions down to a decision.
- It picks each question to **split the data cleanest** (least mixed-up).
- Its superpower: **you can read exactly why** it decided — great for trust.
- Works for categories AND numbers.
- Watch out for **overfitting** (too deep = memorising) — fix with `max_depth`.

Next up — Video 8: Random Forest. One tree can be wrong or overfit — so what if we ask 100 trees and take a vote? The wisdom of the crowd, applied to ML. See you Day 8.